

1. Consider the GBN protocol with a sender window size of 8. Suppose that the sender has sent packets pkt0, pkt1, ..., pkt7 in the pipeline, and pkt6 is lost during transmission (i.e., pkt6 fails to arrive at the receiver). Assume that the medium does not reorder messages.

a) Please list all the ACKs that the sender will receive for the pipelined 8 packets from the receiver.

- A) ACK0, ACK1, ACK2, ACK3, ACK4, ACK5, ACKS
 B) ACK0, ACK1, ACK2, ACK3, ACK4, ACKS, ACKS, ACKS
 C) ACK0, ACK1, ACK2, ACK3, ACK4, ACKS, ACK7
 D) ACK0, ACK1, ACK2, ACK3, ACK4, ACKS, ACK7, ACK7

b) If the SR protocol is running, what will be your answer again?

- A) ACK0, ACK1, ACK2, ACK3, ACK4, ACKS, ACKS
 B) ACK0, ACK1, ACK2, ACK3, ACK4, ACKS, ACKS, ACKS
 C) ACK0, ACK1, ACK2, ACK3, ACK4, ACKS, ACK7
 D) ACK0, ACK1, ACK2, ACK3, ACK4, ACKS, ACK7, ACK7

2. Host A and B are communicating over a TCP connection, and Host B has already received from A all bytes up through byte 126. Next, Host A sends two segments to Host B back-to-back. The first and second segments contain 80 and 40 bytes of data, respectively. In the first segment, the sequence number is 127, the source port number is 302, and the destination port number is 80. Host B sends an acknowledgement whenever it receives a segment from Host A.

a. In the second segment sent from Host A to B, what are the sequence number, source port number, and the destination port number?

- A) 206, 80, 302
 B) 206, 302, 80
 C) 207, 80, 302
 D) 207, 302, 80

b. If the first segment arrives before the second segment, in the acknowledgement of the first arriving segment, what is the acknowledgement number, the source port number, and the destination port number?

- A) 206, 80, 302
 B) 206, 302, 80
 C) 207, 80, 302
 D) 207, 302, 80

c. If the second segment arrives before the first segment, in the acknowledgement of the first arriving segment, what is the acknowledgement number?

- A) 127
 B) 207
 C) 246
 D) 247

d. Suppose the two segments sent by A arrive in order at B. The first acknowledgement is lost and the second acknowledgement arrives after the first timeout interval. Draw a timing diagram, showing these segments and all other segments and acknowledgement sent. (Assume there is no additional packet loss.) For each segment in your figure, provide the sequence number and the number of bytes of data; for each acknowledgement that you add, provide the acknowledgement number.

3. Given values of EstimatedRTT and DevRTT, what value should be used for TCP's timeout interval value?

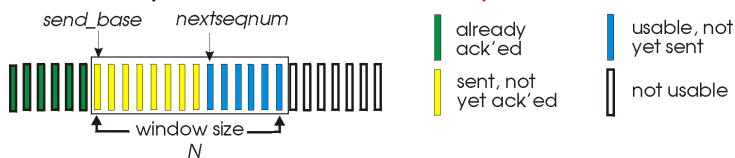
4. Suppose that the three measured SampleRTT values are 106ms, 120 ms, 140 ms. Compute the Estimated RTT after each of these SampleRTT values is obtained, using a value of $\alpha = 0.125$ and assuming that the value of EstimatedRTT was 100 ms just before the first of these three samples were obtained. Compute also the DevRTT after each sample is obtained, assuming a value of $\beta = 0.25$ and assuming the value of DevRTT was 5 ms just before the first of these three samples was obtained. Last, compute the TCP TimeoutInterval after each of these samples is obtained.

What is the value of TimeoutInterval after the three measured SampleRTT?

- A) 135 ms
 B) 147 ms
 C) 150 ms
 D) 164 ms

Go-Back-N: sender

1. **Window:** up to N consecutive unack'ed packets allowed



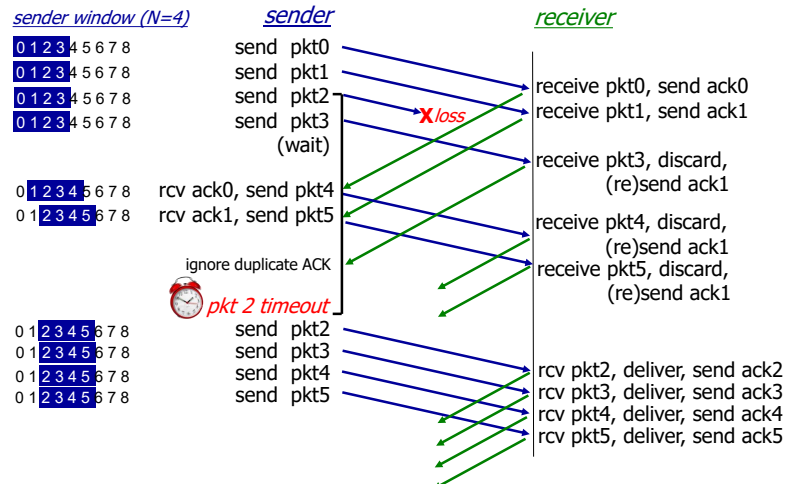
2. **cumulative ACK(n):** all packets with a sequence number up to and including n have been correctly received at the receiver

▪ may receive duplicate ACKs (see receiver)

3. **timer for oldest in-flight pkt (i.e., send_base)**

❖ **timeout:** retransmit packets from send_base to nextseqnum-1 (i.e., yellow region)

GBN in action



Selective repeat

- ❖ receiver *individually* acknowledges all correctly received pkts
 - buffers pkts, as needed, for eventual in-order delivery to upper layer
- ❖ sender only resends pkts for which ACK not received
 - sender timer for each unACKed pkt
- ❖ sender window
 - N consecutive seq #'s
 - limits seq #'s of sent, unACKed pkts

Transport Layer 3-3

Selective repeat

sender

data from above:

- ❖ if next available seq # in window, send pkt

timeout(n):

- ❖ resend pkt n , restart timer

ACK(n) in [sendbase, sendbase+N]:

- ❖ mark pkt n as received
- ❖ if n smallest unACKed pkt, advance window base to next unACKed seq #

receiver

pkt n in [rcvbase, rcvbase+N-1]

- ❖ send ACK(n)
- ❖ out-of-order: buffer
- ❖ in-order: deliver (also deliver buffered, in-order pkts), advance window to next not-yet-received pkt

pkt n in [rcvbase-N, rcvbase-1]

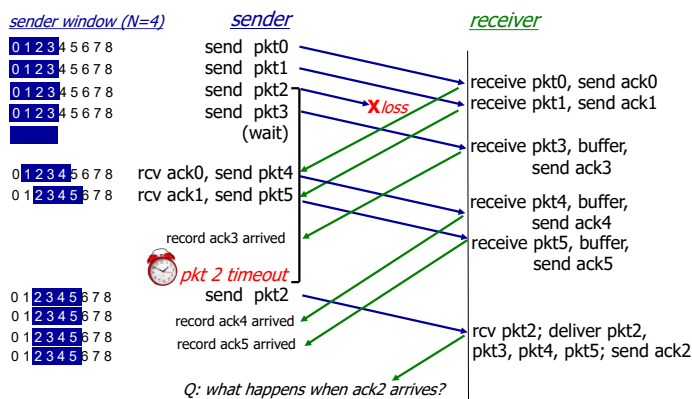
- ❖ ACK(n)

otherwise:

- ❖ ignore

Transport Layer 3-4

Selective repeat in action



Transport Layer 3-5

Problem 1

a) For GBN, the ACK is cumulative. Thus, the ACKs that the sender will receive are ACK0, ACK1, ACK2, ACK3, ACK4, ACK5, ACK5.

Note: For the lost pkt6, the receiver will not send any ACK back. The last ACK5 is the response of receiving pkt7.

b) For SR, only correctly received packets will be individually acknowledged, while all other cases will be ignored. Thus, the ACKs that the sender will receive are ACK0, ACK1, ACK2, ACK3, ACK4, ACK5, ACK7.

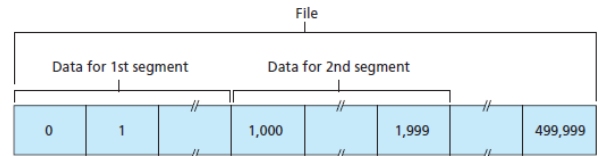
Note: since pkt7 is out-of-order, the receiver will buffer it until pkt6 is eventually correctly received (from retransmission by sender).

TCP payload

- ❖ A TCP header (typically 20-60 bytes long), which includes metadata like source/destination ports, sequence numbers (to track the order of bytes in the data stream), acknowledgment numbers (for confirming receipt), flags (e.g., SYN, ACK, FIN), and checksums for error detection.
- ❖ A payload (the actual data being transferred), which is limited in size by the Maximum Segment Size (MSS). The MSS is the maximum amount of data that can fit into one segment (excluding the header), often negotiated during connection setup to avoid fragmentation. For example, a common MSS is 1,460 bytes on Ethernet networks.

TCP seq. numbers

- ❖ **Maximum segment size (MSS)**: A TCP **payload** (the actual data being transferred), which is limited in size by the Maximum Segment Size (MSS).
- ❖ **Sequence number** for a segment: Byte-stream number of the first byte in the segment
- ❖ E.g., file size = 500,000 bytes and MSS = 1,000 bytes



Dividing file data into TCP segments

TCP ACKs

- ❖ **ACK number that receiver puts in its segment**
= seq # of **next** byte expected from other side
 - E.g., Host A has received all bytes 0 through 535 from B
-> ACK number = 536
 - E.g., Host A has received one segment from Host B containing bytes 0 through 535 and another segment containing bytes 900 through 1,000
-> ACK number = 536
(acknowledges bytes up to the first missing byte of 536)
- ❖ **Cumulative ACK (GBN-style by default)**: TCP acknowledges bytes **up to the first missing byte** in the stream
- ❖ **Q**: how receiver handles **out-of-order** segments
 - A: TCP spec doesn't say, - up to implementor

Problem 2

- TCP treats data as a continuous byte stream, where each segment's sequence number (seq) indicates the **position of its first byte** in that stream.
- The acknowledgment number (ack) in a response segment tells the sender "I've **received everything up to (but not including)** this byte number."
- Host B acknowledges every incoming segment, but the **ack number depends on whether the data is in-order**.

Problem 2

- Given Setup

- Host B has received and acknowledged all bytes 0–126. So, its current "next expected byte" is 127.
- Host A sends Segment 1: 80 bytes of data, seq = 127 (starts with byte 127, covers bytes 127–206).
- Host A sends Segment 2 (back-to-back): 40 bytes of data (covers bytes 207–246).
- Ports in Segment 1: source (A) = 302, destination (B) = 80.

Problem 2.b

If the first segment arrives before the second, what is the ack number, source port, and destination port in the acknowledgment of the first arriving segment (from B to A)?

- Acknowledgment number: Host B receives Segment 1 (bytes 127–206) in order (since it was expecting 127). It now has everything up to byte 206, so it acknowledges the next expected byte: 207.
- Source port: In the ACK (from B to A), ports swap—B's original destination port (80) becomes its source port: 80. Destination port: B sends back to A's original source port: 302.

Correct answer: C) 207, 80, 302

Problem 2.a

Sequence number, source port, and destination port in the second segment (from A to B)

- Sequence number: This is always the byte position of the first byte in the segment. Segment 1 covers bytes 127–206 (80 bytes), so Segment 2 starts right after at byte 207. Thus, seq = 207.
- Source port: This doesn't change—it's still Host A's port, 302.
- Destination port: Still Host B's port, 80.

Correct answer: D) 207, 302, 80

Problem 2.c

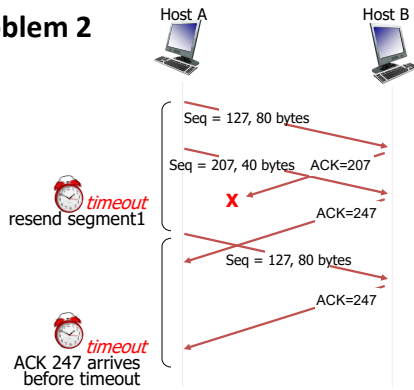
If the second segment arrives before the first, what is the acknowledgment number in the acknowledgment of the first arriving segment (from B to A)?

- Acknowledgment number: Host B receives Segment 2 first (bytes 207–246), but it's out of order—B is still expecting byte 127. TCP buffers out-of-order segments but doesn't advance its ack number until the gap is filled. So, B acknowledges its current next expected byte, which remains 127. (This prompts A to retransmit Segment 1 if needed.)

Correct answer: A) 127 This demonstrates TCP's reliable, ordered delivery: **acks are cumulative**

Problem 2

d)



Timeout Interval for re-transmission

- In TCP, the retransmission timeout (RTO) interval—often called the TimeoutInterval—is adaptively computed to balance **responsiveness** and **stability** in the face of varying network conditions.
 - Given the values of EstimatedRTT (the smoothed estimate of the round-trip time) and DevRTT (the mean deviation of the RTT, capturing variability)

Problem 3

$$\text{TimeoutInterval} = \text{EstimatedRTT} + 4 \cdot \text{DevRTT}$$



↑
estimated RTT

↑
“safety margin”

$$\text{EstimatedRTT} = (1 - \alpha) \cdot \text{EstimatedRTT} + \alpha \cdot \text{SampleRTT}$$

(typically, $\alpha = 0.125$)

$$\text{DevRTT} = (1 - \beta) \cdot \text{DevRTT} + \beta \cdot |\text{SampleRTT} - \text{EstimatedRTT}|$$

(typically, $\beta = 0.25$)

The multiplier of 4 is empirically derived to set the timeout conservatively—about 4 standard deviations above the mean RTT

Problem 4

After obtaining the first sampleRTT 106 ms:

$$\text{EstimatedRTT} = (1 - 0.125) \cdot 100 + 0.125 \cdot 106 = 0.875 \cdot 100 + 13.25 = 100.75 \text{ ms}$$

$$\text{DevRTT} = (1 - 0.25) \cdot 5 + 0.25 \cdot |106 - 100.75| = 0.75 \cdot 5 + 0.25 \cdot 5.25 = 3.75 + 1.3125 = 5.0625 \text{ ms}$$

$$\text{TimeoutInterval} = 100.75 + 4 \cdot 5.0625 = 121 \text{ ms}$$

Problem 4

After obtaining the second sampleRTT 120 ms:

$$\text{- EstimatedRTT} = 0.875 \cdot 100.75 + 0.125 \cdot 120 = 88.15625 + 15 = 103.15625 \text{ ms}$$

$$\text{- DevRTT} = 0.75 \cdot 5.0625 + 0.25 \cdot |120 - 103.15625| = 3.796875 + 4.2109375 = 8.0078125 \text{ ms}$$

$$\text{- TimeoutInterval} = 103.15625 + 4 \cdot 8.0078125 = 103.15625 + 32.03125 = 135.1875 \text{ ms}$$

Problem 4

After obtaining the third sampleRTT 140ms:

$$\text{- EstimatedRTT} = 0.875 \cdot 103.15625 + 0.125 \cdot 140 = 90.26171875 + 17.5 = 107.76171875 \text{ ms}$$

$$\text{- DevRTT} = 0.75 \cdot 8.0078125 + 0.25 \cdot |140 - 107.76171875| = 6.005859375 + 8.0595703125 = 14.0654296875 \text{ ms}$$

$$\text{- TimeoutInterval} = 107.76171875 + 4 \cdot 14.0654296875 = 107.76171875 + 56.26171875 = 164.0234375 \text{ ms}$$